Distributed Global Secondary Index Maintenance for Sharded PostgreSQL

Kunj Bhesaniya (23B0995) Dhruvraj Merchant (23B1041) Dipen Sojitra (23B0913) Harshil Solanki (23B1016)

CS 631 · Checkpoint 1 progress report · 2 November 2026

Problem

When a table is sharded across PostgreSQL servers by primary key, a query on another column (`WHERE email = ?`) must ask every shard, and `UNIQUE(email)` cannot be enforced because each shard sees only its own rows. Spanner, CockroachDB and TiDB solve this with a *global secondary index* (GSI); core PostgreSQL and Citus have none. We build a GSI whose entry for key k lives on node hash(k) mod N, independent of where the row lives. Every write then touches two nodes, so the core challenge is **keeping base table and index consistent under concurrent writes and node crashes**.

Status

The full synchronous GSI works end to end. We planned milestones M1–M6 for this checkpoint and finished M1–M8.

Part	Milestones	Status
Setup: cluster, module, catalog	M1, M2	Done
Reads: plan selection, GsiLookup	M8	Done
Writes: interception, 2PC, uniqueness	M3, M4, M5	Done
Recovery: resolver, background worker	M6, M7	Done
Stress test and benchmarks	M9	Stress test done; benchmarks pending
Online index build, async mode	Stretch	After checkpoint

1 Setup: cluster, connections and GSI catalog

All our code is one PostgreSQL C extension, `gsi_fdw`: a modified copy of `postgres_fdw` (PostgreSQL's wrapper for remote servers) plus about 1,500 lines of our own code.

- **Cluster:** Docker Compose, 1 coordinator and 5 data nodes (PostgreSQL 16). On the coordinator, `users` is hash-partitioned by `id`; each partition is a foreign table on one data node, so PostgreSQL's partition routing sends each row to its shard.
- **Connections:** we reuse postgres_fdw's connection cache and per-node remote transactions, so index writes join the same transaction as the base write.
- **Catalog:** the coordinator stores the nodes (`gsi.gsi_node`) and indexes (`gsi.gsi_index`). `gsi_create('users','email',unique)` creates `gsi_users_email(key, id, shard)` on every node. A table can have several GSIs, one per column.
- **Placement:** hash(k) mod N with the hash PostgreSQL uses for hash partitioning; one C function serves writes, reads and the checker.
- **One entry per row** `(key, id, shard)`: exact primary-key fetches, exact deletes, and a unique B-tree for global uniqueness.

2 Reads: GSI plan selection and query routing

A lookup on the indexed column asks one index node and only the shards that hold matching rows.

- **Plan selection:** a planner hook (`set_rel_pathlist_hook`) sees `WHERE email = <value>` (or any column with a ready GSI) and adds our plan, a **GsiLookup CustomScan**, next to PostgreSQL's "scan all shards" plan (Append). With several matching GSIs it prefers a unique one.
- **Two hops:** (1) hash the key and ask that index node for `(id, shard)`; (2) fetch `WHERE id = ANY(...)` from only those shards.
- **Recheck:** all original conditions are re-evaluated on fetched rows, so a stale index entry can never produce a wrong result.
- **Planner issue solved:** PostgreSQL rebuilds all plans of a partitioned table late in planning (`apply_scanjoin_target_to_paths`), which dropped our plan. After pruning we mark the table non-partitioned for that query, so our plan survives.
- `EXPLAIN` shows `Custom Scan (GsiLookup) on users` with the index node and number of shards fetched. *Limit: the cost is a fixed heuristic for now.*

3 Writes: index maintenance and two-phase commit

Every INSERT, UPDATE and DELETE changes the base row and its entry in every GSI of the table in one distributed transaction, committed atomically with two-phase commit (2PC).

- **Write interception:** for writes that can change an index entry (INSERT, DELETE, UPDATE of an indexed column) we disable postgres_fdw's pushdown and batching, so our per-row hooks run; other UPDATEs keep the pushdown. UPDATE and DELETE also fetch the *old* key of every indexed column, so we know which entries to remove.
- **2PC** (in postgres_fdw's commit callback):
  1. Before commit: `PREPARE TRANSACTION 'gsi_<xid>_<node>_<role>'` on every node involved; any failure aborts everything.
  2. Decision: the coordinator's own local commit (its xid is in every prepared name).
  3. After commit: `COMMIT PREPARED` on **index nodes first, then base shards**, so a visible row always has its index entry.
- **Global uniqueness:** a unique B-tree on the index node; a duplicate aborts the whole transaction, including entries already written to other GSIs. With two concurrent inserts of the same email, exactly one wins.
- **Not supported:** changing `id` (foreign partitions cannot move rows between shards, so we reject it instead of corrupting data).

4 Recovery: finishing transactions after a crash

A crash in the middle of 2PC leaves prepared transactions holding locks. Our resolver finishes them by following the coordinator's recorded decision, as Citus (2PC recovery), CockroachDB (intent resolution) and TiDB (lock resolution) do.

- **Resolver:** reads `pg_prepared_xacts` on each node, asks the coordinator whether the transaction committed (`pg_xact_status`), then runs `COMMIT PREPARED` (index nodes first) or `ROLLBACK PREPARED`.
- **Background worker:** runs the resolver every 5 s inside the coordinator; no manual step.
- **Fault injection:** a test-only setting `gsi.fail_point` crashes the coordinator at an exact protocol step, since real crash windows last microseconds.

Crash point	Result after recovery
After PREPARE, before the decision	Rolled back: row and entry absent
After the coordinator's commit	Rolled forward: row and entry present
Between index commit and base commit	Base finished: row present
Data node paused during commit	Transaction aborts; resolver cleans up

Limitations and plan after Checkpoint 1

Limitations: no global snapshot (during the short commit window a reader may see the entry before the row; the recheck keeps results correct); cross-node deadlocks are ended by a 5 s lock timeout; `gsi_create` needs an empty table; one GSI per column (several per table); only `col = value` uses the GSI (`IN`, ranges and `FOR UPDATE` ask every shard).

Next steps:

1. **Benchmarks:** write throughput and latency with and without the GSI; lookup latency, GSI vs every shard, at 2, 3 and 4 shards.
2. **Cost model:** selectivity-based cost, so the planner uses the GSI only when it is cheaper.
3. **Online index build** on a populated table under live writes (F1 states: write-only, backfill, validate, ready).
4. **Asynchronous mode** via logical decoding, with lag measurement; no global uniqueness in this mode.